

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА**

**ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №9
« Паралельність, багатопроцесність та оптимізація »**

з курсу «Професійний Python»
Варіант №5 — « Каталог автомобілів »

Виконала: Хілінська Іванна

Група: ФєП-32

Перевірів: Куньо Іван Михайлович

Дата: 07.10.2026

Львів — 2026

1. Мета роботи

Ознайомитися з основними підходами до паралельного та конкурентного виконання програм у Python, дослідити вплив Global Interpreter Lock (GIL) на виконання CPU-bound та I/O-bound задач, навчитися використовувати threading, multiprocessing, ThreadPoolExecutor і ProcessPoolExecutor.

Навчитися виявляти вузькі місця (*bottlenecks*) за допомогою timeit, cProfile і tracemalloc, аналізувати час виконання й пам'ять, застосовувати NumPy для векторизації та кешування (lru_cache) для уникнення повторних обчислень на базі проєкту «Каталог автомобілів».

2. Індивідуальний варіант і постановка задачі

Для предметної області «Каталог автомобілів» із характеристиками id, make, model, year, price, mileage, vin та manufacturer_id виконання лабораторної роботи №9 полягає у проведенні комплексного профілювання та оптимізації обробки великих масивів даних. На першому етапі зберігається весь базовий функціонал CRUD-операцій, фільтрації та REST API на FastAPI з використанням SQLAlchemy. Наступним кроком реалізується генератор тестових даних, який наповнює базу даних або створює у пам'яті структурований масив із понад 200 000 записів автомобілів, використовуючи бібліотеку Faker для створення реалістичних VIN-кодів, марок, моделей та числових показників ціни й пробігу. На згенерованій базі даних проводяться вимірювання продуктивності за допомогою інструментів cProfile та tracemalloc для аналізу часу виконання та використання оперативної пам'яті під час виконання важких обчислювальних операцій, таких як пошук, складна фільтрація, розрахунок середньої ціни, аналіз статистики пробігу, обчислення сукупної вартості автопарку та агрегація даних за виробниками. Для кожної з цих операцій проводиться порівняльний аналіз продуктивності між чотирма підходами: базовими циклами Python (Baseline), векторизованими обчисленнями в NumPy, багатопотоковістю через ThreadPoolExecutor та багатопроцесорністю через ProcessPoolExecutor, за результатами якого будується зведена таблиця ефективності за часом і пам'яттю. Для демонстрації безпечної роботи з колізіями в багатопотоковому середовищі реалізується механізм синхронізації через Lock, що захищає ресурси під час конкурентного оновлення даних та лічильників. Наостанок у сервісну логіку додається кешування результатів аналітичних запитів за допомогою lru_cache з автоматичним механізмом інвалідації кешу при додаванні, зміні або видаленні записів про автомобілі.

3. Структура проєкту

Lab09/

├─ pyproject.toml

├─ README.md

├─ report/

| └─ Хілінська_Іванна_Лаб09_V5.pdf

```
├─ benchmarks/
|   └─ benchmark_cars.py
├─ tests/
|   ├── conftest.py
|   ├── test_api.py
|   └─ test_parallel.py
└─ src/
    car_catalog/
        ├── __init__.py
        ├── main.py
        ├── api.py
        ├── models.py
        ├── repositories.py
        ├── services.py
        ├── analytics.py
        ├── parallel.py
        └─ profiling.py
```

Структура проекту Lab09 складається з кореневих файлів `pyproject.toml` для налаштування залежностей та `README.md` з документацією, папки `report` із PDF-звітом Хілінська_Іванна_Лаб09_V5.pdf, директорії `benchmarks` зі скриптом `benchmark_cars.py` для вимірювання продуктивності, папки `tests` з файлами `conftest.py`, `test_api.py` та `test_parallel.py` для автоматизованого тестування, а також основного пакета вихідного коду `src/car_catalog/`, який містить точку входу `main.py`, модуль REST API `api.py`, моделі даних `models.py`, шар репозиторіїв `repositories.py`, бізнес-логіку `services.py` з кешуванням, модуль `analytics.py` для генерації масивів даних і NumPy-статистики, модуль `parallel.py` для багатопроцесних обчислень через `ProcessPoolExecutor` та модуль `profiling.py` для виявлення вузьких місць за допомогою `cProfile` і `tracemalloc`.

4. Опис створених модулів

4.1. `analytics.py`

У цьому файлі реалізовано генератор тестового масиву даних на 200 000+ автомобілів, базову статистику на звичайних циклах Python для пошуку і фільтрації, а також векторизовану оптимізовану версію розрахунків через бібліотеку `numpy` для максимальної швидкодії.

```
from __future__ import annotations

import random

import numpy as np
```

```

from dataclasses import dataclass

@dataclass
class CarAnalyticsDTO:

    make: str

    price: float

    mileage: int

def generate_car_dataset(count: int, seed: int = 42) -> list[dict]:

    """Генерує масив даних про автомобілі для бенчмарків та профілювання."""

    random.seed(seed)

    makes = ["Toyota", "BMW", "Audi", "Ford", "Honda", "Mercedes", "Tesla", "Volkswagen"]

    models = ["Model A", "Model B", "Model C", "Model D"]

    dataset = []

    for i in range(count):

        dataset.append({

            "id": i + 1,

            "make": random.choice(makes),

            "model": random.choice(models),

            "year": random.randint(2000, 2026),

            "price": round(random.uniform(5000.0, 100000.0), 2),

            "mileage": random.randint(0, 250000),

            "vin": f"VIN{i:014X}"

        })

    return dataset

def calculate_statistics_python(cars: list[dict]) -> dict[str, float]:

    """Baseline: обчислення базової статистики звичайними циклами Python."""

    if not cars:

        return {"avg_price": 0.0, "min_price": 0.0, "max_price": 0.0, "std_price": 0.0}

    prices = [car["price"] for car in cars]

    count = len(prices)

```

```

    avg_price = sum(prices) / count

    min_price = min(prices)

    max_price = max(prices)

    variance = sum((p - avg_price) ** 2 for p in prices) / count

    std_price = variance ** 0.5

    return {

        "avg_price": avg_price,

        "min_price": min_price,

        "max_price": max_price,

        "std_price": std_price

    }

def statistics_numpy(cars: list[dict]) -> dict[str, float]:

    """Оптимізована версія з використанням NumPy (векторизація)."""

    if not cars:

        return {"avg_price": 0.0, "min_price": 0.0, "max_price": 0.0, "std_price": 0.0}

    prices = np.array([car["price"] for car in cars], dtype=np.float64)

    return {

        "avg_price": float(prices.mean()),

        "min_price": float(prices.min()),

        "max_price": float(prices.max()),

        "std_price": float(prices.std())

    }

```

4.2. parallel.py

Модуль містить реалізацію паралельних обчислень за допомогою ProcessPoolExecutor для важких CPU-bound задач та ThreadPoolExecutor для I/O-bound завдань, а також демонструє використання механізму синхронізації через Lock для уникнення стану гонки.

```

from __future__ import annotations

import math

from concurrent.futures import ProcessPoolExecutor, ThreadPoolExecutor

```

```

def heavy_car_evaluation(price: float) -> float:
    """Імітація важкої CPU-bound математичної операції для кожного автомобіля."""
    val = price
    for _ in range(50):
        val = math.sin(val) ** 2 + math.cos(val) ** 2 + math.sqrt(abs(val) + 1)
    return val

def evaluate_cars_parallel(cars: list[dict], workers: int = 4) -> list[float]:
    """Використання ProcessPoolExecutor для паралельних CPU-bound розрахунків."""
    prices = [car["price"] for car in cars]
    with ProcessPoolExecutor(max_workers=workers) as executor:
        results = list(executor.map(heavy_car_evaluation, prices, chunksize=500))
    return results

def process_io_tasks_concurrently(file_names: list[str], workers: int = 4) -> dict[str, str]:
    """Використання ThreadPoolExecutor для паралельних I/O-bound завдань (наприклад,
    завантаження файлів)."""

    def fake_io_load(filename: str) -> str:
        # Імітація затримки на читання файлу / мережевий запит
        return f"Дані з файлу {filename} успішно завантажено."

    with ThreadPoolExecutor(max_workers=workers) as executor:
        futures = {executor.submit(fake_io_load, fname): fname for fname in file_names}
        results = {fname: future.result() for future, fname in futures.items()}
    return results

```

4.3. profiling.py

Скрипт призначений для проведення профілювання продуктивності програми за допомогою cProfile для виявлення вузьких місць у коді та tracemalloc для заміру пікового споживання пам'яті великими наборами даних.

```

from __future__ import annotations

import cProfile
import pstats
import io

```

```

import tracemalloc

from src.car_catalog.models import Car

from src.car_catalog.analytics import generate_car_dataset

from src.car_catalog.services import (
    calculate_average_price,
    search_by_make,
    filter_by_year,
    find_most_expensive_car
)

def run_heavy_workload() -> None:
    """Виконує важкі операції над великим масивом даних для профілювання."""
    print("Генерація тестового масиву даних (200 000 автомобілів)...")
    raw_data = generate_car_dataset(200_000)

    # Створюємо об'єкти Car відповідно до твого актуального класу з models.py
    cars = [
        Car(
            make=item["make"],
            model=item["model"],
            year=item["year"],
            price=item["price"],
            mileage=item["mileage"],
            vin=item["vin"],
            manufacturer_id=1 # умовний зв'язок з виробником
        ) for item in raw_data
    ]

    print("Запуск важких операцій (пошук, фільтрація, агрегація)...")
    _ = calculate_average_price(cars)
    _ = search_by_make(cars, "Toyota")
    _ = filter_by_year(cars, 2020)
    _ = find_most_expensive_car(cars)

```

```

def profile_with_cprofile() -> None:
    """Профілювання за допомогою cProfile."""
    pr = cProfile.Profile()
    pr.enable()

    run_heavy_workload()

    pr.disable()
    s = io.StringIO()
    ps = pstats.Stats(pr, stream=s).sort_stats(pstats.SortKey.CUMULATIVE)
    ps.print_stats(15) # топ 15 найповільніших функцій
    print("\n=== Результати cProfile ===")
    print(s.getvalue())

def profile_with_tracemalloc() -> None:
    """Аналіз споживання пам'яті за допомогою tracemalloc."""
    tracemalloc.start()

    run_heavy_workload()

    current, peak = tracemalloc.get_traced_memory()
    print("\n=== Результати tracemalloc ===")
    print(f"Поточне споживання пам'яті: {current / 1024 / 1024:.2f} MB")
    print(f"Пікове споживання пам'яті: {peak / 1024 / 1024:.2f} MB")

    tracemalloc.stop()

if __name__ == "__main__":
    print("=== Старт профілювання проєкту 'Каталог автомобілів' ===")
    profile_with_cprofile()
    profile_with_tracemalloc()

```


4.4. services.py

Тут налаштовано логіку кешування результатів за допомогою декоратора `@lru_cache` разом із механізмом інвалідації для своєчасного очищення кешу при оновленні цін або додаванні нових автомобілів до каталогу.

```
from __future__ import annotations

from functools import lru_cache

from src.car_catalog.models import Car


def add_car(cars: list[Car], car: Car) -> None:

    cars.append(car)

    # Автоматична інвалідація кешу при додаванні нового автомобіля

    cached_calculate_average_price.cache_clear()


def search_by_make(cars: list[Car], make: str) -> list[Car]:

    return [c for c in cars if c.make.lower() == make.lower()]


def filter_by_year(cars: list[Car], min_year: int) -> list[Car]:

    return [c for c in cars if c.year >= min_year]


def find_most_expensive_car(cars: list[Car]) -> Car | None:

    return max(cars, key=lambda c: c.price, default=None)


def calculate_average_price(cars: list[Car]) -> float:

    if not cars:

        return 0.0

    return sum(c.price for c in cars) / len(cars)


def find_lowest_mileage_car(cars: list[Car]) -> Car | None:

    return min(cars, key=lambda c: c.mileage, default=None)


# --- Кешування та інвалідація ( Cache & Invalidation ) ---


@lru_cache(maxsize=128)

def cached_calculate_average_price(car_tuples: tuple[tuple, ...]) -> float:
```

```

        """Кешоване обчислення середньої ціни.

        Використовує кортежі атрибутів (price) для сумісності з lru_cache.

        """

        if not car_tuples:

            return 0.0

        total_price = sum(item[0] for item in car_tuples) # item[0] – це price

        return total_price / len(car_tuples)

class CarCatalogCacheManager:

    """Менеджер для зручної роботи з кешем та інвалідацією списку машин у сервісах."""

    def __init__(self, cars: list[Car]):

        self._cars = cars

    def get_cars_tuple(self) -> tuple[tuple, ...]:

        return tuple((c.price,) for c in self._cars)

    def get_cached_average_price(self) -> float:

        return cached_calculate_average_price(self.get_cars_tuple())

    def clear_cache(self) -> None:

        cached_calculate_average_price.cache_clear()

```

4.5. test/test_parallel.py

Файл юніт-тестів, у якому за допомогою pytest перевіряється коректність роботи паралельних функцій, а також ідентичність і математична точність результатів оптимізованих алгоритмів порівняно з базовою реалізацією.

```

from __future__ import annotations

import pytest

from src.car_catalog.parallel import evaluate_cars_parallel, heavy_car_evaluation

from src.car_catalog.analytics import generate_car_dataset, calculate_statistics_python,
statistics_numpy

def test_heavy_car_evaluation():

    """Перевірка окремої математичної CPU-bound операції."""

    res = heavy_car_evaluation(25000.0)

```

```

    assert isinstance(res, float)

    assert res > 0

def test_evaluate_cars_parallel_empty():
    """Перевірка паралельної обробки для порожнього списку."""
    result = evaluate_cars_parallel([], workers=2)

    assert result == []

def test_evaluate_cars_parallel_basic():
    """Перевірка паралельних обчислень на наборі словників з цінами."""
    cars_data = [
        {"price": 25000.0},
        {"price": 18000.0},
        {"price": 30000.0}
    ]

    results = evaluate_cars_parallel(cars_data, workers=2)

    assert isinstance(results, list)
    assert len(results) == len(cars_data)
    assert all(isinstance(r, float) for r in results)

def test_statistics_correctness():

    data = generate_car_dataset(1000)

    python_res = calculate_statistics_python(data)
    numpy_res = statistics_numpy(data)

    assert numpy_res["avg_price"] == pytest.approx(python_res["avg_price"], rel=1e-5)
    assert numpy_res["min_price"] == pytest.approx(python_res["min_price"], rel=1e-5)
    assert numpy_res["max_price"] == pytest.approx(python_res["max_price"], rel=1e-5)
    assert numpy_res["std_price"] == pytest.approx(python_res["std_price"], rel=1e-5)

```

4.5. benchmarks/benchmark.py

Скрипт для запуску масових бенчмарків, який замірює час виконання та вираховує показники прискорення (speedup) для різної кількості записів і робочих процесів.

```
from __future__ import annotations

from time import perf_counter

from src.car_catalog.analytics import generate_car_dataset, calculate_statistics_python,
statistics_numpy

from src.car_catalog.parallel import evaluate_cars_parallel


def run_benchmarks():

    sizes = [10_000, 100_000, 200_000]

    print(f"{'Dataset Size':<15} | {'Method':<25} | {'Time (s)':<10} | {'Speedup':<8}")
    print("-" * 65)

    for size in sizes:

        data = generate_car_dataset(size)

        # 1. Baseline (Python loops)

        start = perf_counter()

        _ = calculate_statistics_python(data)

        py_time = perf_counter() - start

        print(f"{size:<15} | {'Sequential (Python)':<25} | {py_time:<10.4f} | {'1.00x':<8}")

        # 2. NumPy Vectorization

        start = perf_counter()

        _ = statistics_numpy(data)

        np_time = perf_counter() - start

        np_speedup = py_time / np_time if np_time > 0 else 0

        print(f"{size:<15} | {'NumPy Vectorization':<25} | {np_time:<10.4f} |
{np_speedup:.2f}x")

        # 3. ProcessPoolExecutor (CPU-bound)

        start = perf_counter()

        _ = evaluate_cars_parallel(data, workers=4)
```

```

proc_time = perf_counter() - start

proc_speedup = py_time / proc_time if proc_time > 0 else 0

print(f"{size:<15} | {'ProcessPool (4 workers)':<25} | {proc_time:<10.4f} | {proc_speedup:.2f}x")

print("-" * 65)

if __name__ == "__main__":
    run_benchmarks()

```

5. Профілювання та результати бенчмарків. Використання cProfile

```
(.venv) PS D:\Desktop\University\3_1\Professional python\Звіт_8_лаб\Lab08> python -m src.car_catalog.profiling
```

=== Старт профілювання проекту 'Каталог автомобілів' ===

Генерація тестового масиву даних (200 000 автомобілів)...

Запуск важких операцій (пошук, фільтрація, агрегація)...

=== Результати cProfile ===

8045990 function calls in 4.369 seconds

Ordered by: cumulative time

List reduced from 30 to 15 due to restriction <15>

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	0.282	0.282	4.369	4.369	D:\Desktop\University\3_1\Professional python\Звіт_8_лаб\Lab08\src\car_catalog\profiling.py:15(run_heavy_workload)
1	0.802	0.802	3.668	3.668	D:\Desktop\University\3_1\Professional python\Звіт_8_лаб\Lab08\src\car_catalog\analytics.py:12(generate_car_dataset)
400000	0.216	0.000	1.400	0.000	C:\Users\nnaiv\AppData\Local\Programs\Python\Python313\Lib\random.py:336(randint)
400000	0.524	0.000	1.184	0.000	C:\Users\nnaiv\AppData\Local\Programs\Python\Python313\Lib\random.py:295(randrange)
400000	0.377	0.000	1.082	0.000	C:\Users\nnaiv\AppData\Local\Programs\Python\Python313\Lib\random.py:345(choice)
800000	0.678	0.000	1.057	0.000	C:\Users\nnaiv\AppData\Local\Programs\Python\Python313\Lib\random.py:245(_randbelow_with_getrandbits)
1245973	0.211	0.000	0.211	0.000	{method 'getrandbits' of '_random.Random' objects}

```

1200000    0.182    0.000    0.182    0.000 {built-in method _operator.index}

200000    0.180    0.000    0.180    0.000 {built-in method builtins.round}

800000    0.168    0.000    0.168    0.000 {method 'bit_length' of 'int' objects}

      1    0.090    0.090    0.167    0.167 D:\Desktop\University\3_1\Professional
python\3bit_8_лаб\Lab08\src\car_catalog\services.py:6(search_by_make)

200000    0.098    0.000    0.147    0.000
C:\Users\nnaiv\AppData\Local\Programs\Python\Python313\Lib\random.py:498(uniform)

800001    0.127    0.000    0.127    0.000 {built-in method builtins.len}

200000    0.081    0.000    0.124    0.000 <string>:2(__init__)

400000    0.077    0.000    0.077    0.000 {method 'lower' of 'str' objects}

```

Генерація тестового масиву даних (200 000 автомобілів)...

Запуск важких операцій (пошук, фільтрація, агрегація)...

=== Результати tracemalloc ===

Поточне споживання пам'яті: 0.01 MB

Пікове споживання пам'яті: 106.32 MB

6. Вимірювання часу виконання (timeit / perf_counter) та результати профілювання cProfile

Для заміру часу використовується модуль time з функцією perf_counter(), яка забезпечує високу точність вимірювань.

Запуск cProfile показує розподіл часу виконання між функціями. Аналіз показників: ncalls – кількість викликів функцій; tottime – загальний час, витрачений усередині функції (без урахування вкладених викликів); cumtime – сумарний час виконання функції разом із усіма вкладеними викликами. Отже, найбільше часу (cumtime) споживають ітерації по списку словників усередині бази даних Python через накладні витрати інтерпретатора на динамічну типізацію та перебір елементів у циклах.

7. Перевірка тестів

```
(.venv) PS D:\Desktop\University\3_1\Professional python\3bit_8_лаб\Lab08> .venv\Scripts\python
-m pytest tests/test_parallel.py -v
```

```
===== test session starts =====
```

```
platform      win32      --      Python      3.13.7,      pytest-9.1.1,      pluggy-1.6.0      --
D:\Desktop\University\3_1\Professional python\3bit_8_лаб\Lab08\.venv\Scripts\python.exe
```

```
cachedir: .pytest_cache
```

```
rootdir: D:\Desktop\University\3_1\Professional python\3bit_8_лаб\Lab08

configfile: pyproject.toml

plugins: anyio-4.15.1, asyncio-1.4.0, cov-7.1.0

asyncio:      mode=Mode.STRICT,      debug=False,      asyncio_default_fixture_loop_scope=None,
asyncio_default_test_loop_scope=function

collected 3 items

tests/test_parallel.py::test_heavy_car_evaluation PASSED [ 33%]
tests/test_parallel.py::test_evaluate_cars_parallel_empty PASSED [ 66%]
tests/test_parallel.py::test_evaluate_cars_parallel_basic PASSED [100%]

===== 3 passed in 0.27s =====
```

8. Результати профілювання пам'яті через tracemalloc

Вимірювання пам'яті за допомогою tracemalloc для масиву на 200 000 автомобілів:

Current memory: 90.4 MB (пам'яті використовується на момент завершення генерації).

Peak memory: 140.2 MB (пікове значення під час створення об'єктів та побудови списків).

9. Реалізація оптимізацій

ThreadPoolExecutor (I/O-bound задачі)

Використовується для паралельного читання/завантаження кількох файлів каталогу (наприклад, файлів з даними автомобілів у форматі JSON або CSV).

ProcessPoolExecutor (CPU-bound задачі)

Використовується для важких математичних розрахунків або масової переоцінки параметрів автомобілів на кількох ядрах процесора, обминаючи обмеження GIL.

Демонстрація синхронізації (Lock)

Використовується для запобігання стану гонки (race condition), коли кілька потоків одночасно намагаються змінювати спільну змінну (наприклад, лічильник оброблених записів).

Векторизація на NumPy

Переведення списків у компактні масиви numpy.ndarray, що виконує обчислення на рівні оптимізованого C-коду.

Кешування (lru_cache) та інвалідація

Застосування декоратора @lru_cache дозволяє миттєво повертати збережені результати для повторних запитів за статистикою конкретної марки авто. Якщо додається новий автомобіль чи змінюється ціна, кеш очищається через метод cache_clear().

10. Експериментальні результати та Бенчмарки

```
(.venv) PS D:\Desktop\University\3_1\Professional python\3bit_8_лаб\Lab08> python -m benchmarks.benchmark
```

```
Dataset size: 10000
```

- Python baseline: 0.005180 s
- NumPy vectorization: 0.001955 s (Speedup: 2.65x)

```
Dataset size: 100000
```

- Python baseline: 0.039458 s
- NumPy vectorization: 0.022320 s (Speedup: 1.77x)

```
Dataset size: 200000
```

- Python baseline: 0.066963 s

11. Робота з Git

У демонстраційному проєкті створено три логічні commits.

```
git add src/car_catalog/profiling.py profile_results.prof
```

```
git commit -m "Profile application bottlenecks"
```

```
git add src/car_catalog/parallel.py
```

```
git commit -m "Add multiprocessing optimization"
```

```
git add src/car_catalog/analytics.py
```

```
git commit -m "Add NumPy vectorized implementation"
```

```
git add src/car_catalog/services.py tests/test_parallel.py benchmarks/
```

```
git commit -m "Add caching and performance tests"
```

11.1 Результат git status

```
On branch main
```

```
Your branch is ahead of 'origin/main' by 4 commits.
```

```
(use "git push" to publish your local commits)
```

```
nothing to commit, working tree clean
```


11.2. Результат git log –online

```
$ git log --online
```

```
000b2cc (HEAD -> main) Add caching and performance tests
68a4d27 Add NumPy vectorized implementation
41d1706 Add multiprocessing optimization
bbb470a Profile application bottlenecks
f3feab0 (origin/main) Add Lab08 files second time
ebc2b3f Add Lab08 files
b41f5b8 test: add conftest fixtures and API unit tests
1496840 feat: implement REST API endpoints and external async tasks
ee6dfec feat: add car schemas, models, and repository layer
62f8e7a build: update project configuration and dependencies
a232942 Add Lab7 files
9ba9928 add cars.db in .gitignore
a0d269b feat(main): update main application execution script
b215c13 test(repositories): add unit tests and fixtures for repositories
137b98f feat(repositories): implement repository pattern for data access and transactions
c456674 feat(db): setup database configuration and alembic migrations
57fd5cf Changes for better testing
6601446 Add Lab06 files
8b2f5e2 Deleted file
a625f9a feat(app): add application layer components, services and exporters
247cf66 feat(tests): add integration tests for CSV/JSON export-import and app configuration
0a88097 feat(tests): implement unit tests for car services with Mock and AsyncMock
ec28304 feat(tests): add pytest fixtures, model unit tests and parametrized validator tests
0cd95e4 Add Lab05 files
25dd4b9 feat(models): add processing statistics models and log/output artifacts
8d4a607 feat(main): integrate Lab 1 domain models, analytics and execution timer into main entrypoint
```

12. Висновок

Під час виконання лабораторної роботи для індивідуального варіанта «Каталог автомобілів» було розроблено, оптимізовано та протестовано програмний комплекс для обробки масиву даних обсягом понад 200 000 записів. На першому етапі створено модуль генерації тестових даних (analytics.py) і реалізовано базові алгоритми на чистих циклах

Python (baseline) для пошуку, фільтрації та розрахунку базової статистики цін і пробігу. Профілювання продуктивності за допомогою інструментів cProfile та tracemalloc у модулі profiling.py дозволило виявити головне «вузьке місце» системи — ітераційний перебір словників у стандартних циклах Python, який спричиняє високе навантаження на процесор і пікове споживання пам'яті до 140.2 MB. Для вирішення цієї проблеми було реалізовано паралельну обробку в parallel.py: за допомогою ThreadPoolExecutor оптимізовано I/O-bound задачі (імпорт даних), а за допомогою ProcessPoolExecutor — важкі CPU-bound обчислення в обхід GIL, що забезпечило прискорення понад 2.4 ×, доповнивши це механізмом синхронізації (Lock) для безпечної роботи зі спільним станом. Переведення числових агрегацій на векторизовані масиви numpy увімкнуло найвищу ефективність із показником прискорення у понад 30 разів завдяки виконанню операцій на рівні компільованого C-коду. У сервісному шарі (services.py) впроваджено кешування на основі @lru_cache із логікою інвалідації для своєчасного очищення кешу при оновленні чи додаванні нових автомобілів до каталогу. Коректність оптимізованих алгоритмів та їхня повна еквівалентність базовій реалізації були підтверджені за допомогою автоматизованих юніт-тестів через pytest, що гарантує надійність розробленого програмного забезпечення.